Repair ledger — Aletheia -> ODIN Bridge

Issues found and fixed during the build, in Project Witness's "make claims, attack
claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): Aletheia -> ODIN Bridge

First bridge build — Move 13 of Project Witness, and the first component that
connects two frozen pieces (Aletheia v0.1 and ODIN v0.1). It converts a verified
Aletheia Evidence Bundle into an ODIN claim set containing only integrity/presence
claims, writes a deterministic output folder, and verifies that output later. It is
not connected to the Companion workflow yet.

Scope held deliberately narrow: one bundle -> one claim set; integrity/presence
claims only; no multiple files, directories, web, sensors, OCR, LLM, natural

language, truth claims, signing, or anchoring. The bridge converts integrity
metadata only — it never interprets evidence contents or asserts their truth.

Design decisions worth recording:

- **Truth-claim guard at two layers.** The claim builder refuses to emit any
  predicate outside the two allowed ones, and additionally refuses any predicate
  containing a truth/authenticity token (true, truth, authentic, trusted, proven,
  real, genuine, accurate, content). `verify` independently re-checks the output for disallowed/truth-like predicates (`DISALLOWED_PREDICATE` / `TRUTH_PREDICATE`), so a hand-injected `contents.true` claim is caught even after checksums are resynced.
- **Verify recomputes cross-references; it does not trust the report.** It confirms
  report `case_id` == claims `case_id`, report `evidence_id` == manifest `evidence_id`, every support `source_sha256` == manifest `bundled_file_sha256`,
  and report `odin_claim_count` == actual count — each independently, after a
  canonical + hash check. Resyncing the checksums after a lie does not help; the semantic checks still fail.

- **Frozen boundaries are injectable.** Aletheia verify and ODIN validate are called via subprocess at runtime, but both are injected as in-process fakes in the unit suite, so the freeze gate spawns no Aletheia/ODIN subprocesses (the subprocess-heavy path is exercised only by the documented manual integration smoke test). `--odin-root` is optional; when omitted, validation is `SKIPPED`.
- **No evidence custody.** v0.1 copies the manifest, not the evidence file, by design — a test asserts no `evidence/` dir or `.bin` is written.
- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output directory; `require()` not bare assert; strict checksums; unknown-file rejection; in-process selfcheck that never launches the unit suite.

Real end-to-end integration confirmed. Against frozen Aletheia v0.1 and ODIN v0.1: built a real bundle, `convert` produced `BRIDGE CONVERTED` with `odin_validation_status = VALID` and exactly `evidence.present` + `evidence.integrity_verified`; `bridge verify`

passed; ODIN `reason` over the
generated claims returned `CONSISTENT`; ODIN
`verify` passed.

Release gate (Coordinator v0.1.8 / Companion v0.1 / Aletheia v0.1 / ODIN v0.1
lesson applied directly): no subprocess orchestrator in the freeze gate. The
authoritative gate is the explicit command list in RELEASE_COMMANDS.md plus the
static `verify_release_manifest.py` (launches no subprocesses; additionally checks
the README scopes the bridge to integrity metadata only and disclaims proving
truth).

Verified: all release-gate commands pass individually from a clean extraction
(see TEST_RESULTS.md); selfcheck 8/8 in-process gates; verify_release_manifest.py
passes (8/8, no subprocesses); tamper to any output file, the checksums, the file
set, the report's cross-references, the support SHA, or an injected truth predicate
is detected with the correct reason.